How about the last of the three favorites of JavaScript programmers: `reduce`?

### 7.5.3. reduce

Imagine you're a loan shark—people owe you money. How do you calculate the total amount you're owed? An array of two people who owe you money is easy to add up just by looking at it or by explicitly adding the two values:

```
friends = [
  { name: 'bob', owes: 10 }
  { name: 'sam', owes: 15 }
]

total = friends[0].owes + friends[1].owes
# 25
```

What if you loaned money to a thousand people? You'd use a comprehension:

```
owed = 0
for friend in friends
  owed += friend.owes
```

The built-in `Array::reduce` also works:

```
owing = (initial, friend) ->
  if initial.owers then initial.owes + friend.owes
owed = friends.reduce owing
```

It isn't clear if the comprehension is better. Comprehensions are useful, but they're not the only technique to consider. Comprehensions also have some gotchas you need to be aware of. The first involves functions.

### 7.5.4. Defining functions inside comprehensions

When using comprehensions, you need to be careful about scoping because there's a mistake you can easily make even when you know how